Auditoría, Calidad y Fiabilidad Informáticas

Universidad Complutense de Madrid Curso 2026/27

Proyecto final: especificación y verificación de una arquitectura real en Maude

---

Este proyecto se realiza en grupos de dos o tres estudiantes. Cada grupo elegirá una arquitectura, protocolo o aplicación *real* (no un ejemplo de juguete inventado para la ocasión), la entenderá en profundidad y la especificará en Maude, demostrando propiedades sobre ella.

Ejercicio 1 Elegid un sistema real y estudiadlo lo suficiente como para poder explicar, en vuestras propias palabras, cómo funciona. Podéis elegir uno de la siguiente lista, o proponer otro (justificando su interés y con aprobación previa del profesor).

Junto a cada sistema se indica el tamaño de equipo para el que está pensado su alcance: **[2]** encaja bien en una pareja, **[3]** pide un grupo de tres y **[2–3]** funciona en ambos casos. La lista va de más a menos exigente. Es una recomendación, no una restricción, y **elegir un sistema marcado [2] no limita la nota**: lo que se valora es la profundidad del desarrollo en relación con el tamaño del equipo.

- **Consenso simplificado tipo Raft** (elección de líder más replicación de un *log*): como mucho hay un líder por término; las entradas confirmadas no se pierden ante el fallo de una minoría de nodos **[3]**.
- **Confirmación en dos fases** (*two-phase commit*, 2PC): todos los participantes deciden lo mismo (atomicidad); ausencia de bloqueo indefinido si el coordinador falla tras la fase de *prepare* **[3]**.
- **Gestor de bloqueos distribuido** (*distributed lock manager*): exclusión mutua entre clientes, ausencia de interbloqueo, liberación del bloqueo tras el fallo de su titular **[3]**.
- **Sistema de mensajería con garantías de entrega** (*at-least-once* o *exactly-once*): todo mensaje enviado se entrega al menos una vez; ausencia de duplicados cuando se exige entrega exacta **[3]**.
- **Caché con invalidación** (*write-through* o *write-back*): coherencia entre la caché y el almacenamiento persistente, ausencia de lecturas obsoletas tras una invalidación **[2–3]**.
- **Saludo inicial de un protocolo real** (*handshake*; por ejemplo el saludo de tres vías de TCP, o una versión simplificada de TLS): siempre termina en un estado consistente en ambos extremos **[2–3]**.
- **Elección de líder** (algoritmo *bully* u otro similar): se elige un único líder, y es siempre el de mayor prioridad entre los nodos vivos **[2–3]**.
- **Protocolo de cotilleo** (*gossip protocol*): la información se propaga eventualmente a todos los nodos vivos; convergencia del estado **[2–3]**.
- **Productor-consumidor con cola acotada**: ausencia de pérdida de mensajes, ausencia de desbordamiento del *buffer*, ausencia de inanición del consumidor **[2]**.
- **Limitador de peticiones** (*rate limiter*, con *token bucket* o *leaky bucket*): nunca se aceptan más peticiones que el límite configurado en una ventana de tiempo; el sistema se recupera tras un pico de tráfico **[2]**.
- **Cortacircuitos** (*circuit breaker*, patrón de resiliencia en microservicios): el circuito se abre tras N fallos consecutivos y no se realizan llamadas al servicio caído mientras el circuito está abierto **[2]**.
- **Balanceador de carga** (*round robin* o *least connections*): reparto justo de peticiones entre los nodos activos; ningún nodo caído recibe tráfico **[2]**.

En la memoria debéis explicar qué mecanismos esenciales del sistema real vais a modelar y, explícitamente, qué se simplifica o se deja fuera y por qué esa simplificación no desvirtúa las propiedades que queréis estudiar. **La profundidad del desarrollo del sistema elegido se tendrá en cuenta en la evaluación**: un sistema con más concurrencia, más casos límite o propiedades más difíciles de expresar y verificar se valorará más.

Ejercicio 2 Especificad el sistema elegido en Maude. Definid los tipos y estructuras de datos que creáis necesarias, elegid bien si algo debe escribirse como una ecuación o como una regla, y dividid el problema en módulos (módulos funcionales para los tipos de datos del dominio, módulo de sistema para la dinámica del protocolo). Comentad el código e incluid instrucciones claras para cargarlo.

Escribid también al menos un término de ejemplo con un número razonable de agentes/nodos/mensajes en el sistema (justificad por qué ese ejemplo es suficientemente representativo, y no un caso degenerado).

Ejercicio 3 Definid, usando lo que hayáis definido en el apartado 2, al menos dos propiedades relevantes del sistema (por ejemplo: exclusión mutua, ausencia de interbloqueo, entrega en orden, ausencia de pérdida de mensajes, terminación, ausencia de inanición). Evitad propiedades triviales que no digan nada relevante del sistema.

Ejercicio 4 Comprobad si las propiedades del apartado 3 se cumplen:

- Usad el comando `search` para explorar el espacio de estados y comprobar (o refutar) la alcanzabilidad de estados no deseados.
- Formulad al menos una propiedad temporal (LTL) y comprobadla con el model checker de Maude, sobre un módulo que incluya `MODEL-CHECKER`.

Incluid en la memoria la salida real de Maude (transcripción literal de los comandos y sus resultados, no solo una descripción de lo que "debería" pasar) y explicad brevemente el resultado del model checker en cada caso.

Ejercicio 5 Introducid de forma deliberada, sobre vuestra propia especificación, al menos **dos mutaciones** que rompan alguna de las propiedades que habéis verificado en el apartado 4 (por ejemplo: eliminar una condición de guarda, invertir el orden de dos reglas, quitar un caso de un patrón). Volved a ejecutar `search` o el model checker sobre cada mutante y mostrad que la propiedad ahora falla, incluyendo el contraejemplo o el estado ofensivo encontrado por Maude.

Explicad, en términos del sistema real (no solo del código Maude), qué error concreto representa cada mutación: por ejemplo, "esto simula qué pasaría si el titular de un bloqueo cae sin liberarlo".

Ejercicio 6 Escribid qué reglas podrían transformarse en ecuaciones para optimizar el model checking de cada propiedad del apartado 4 (basta con nombrar las reglas a cambiar, no hace falta modificar el código).

Entrega. Además del código Maude, entregad una memoria que integre los apartados anteriores en un documento coherente. **El proyecto se corregirá con Maude 3.5**: podéis trabajar con la versión que queráis, pero comprobad antes de entregar que vuestra especificación carga sin avisos y da los mismos resultados en esa.

Incluid en la memoria un **reparto de tareas**: una tabla que diga, para cada uno de los apartados 1–6, quién lo lideró, más un porcentaje aproximado de contribución por integrante que sume 100. Lo acuerda y lo firma todo el equipo. No hace falta que sea equilibrado: un reparto desigual declarado con honestidad no penaliza, y uno maquillado se nota en la defensa.

Defensa. Cada equipo defenderá su proyecto oralmente en horario de clase, con preguntas individualizadas a cada integrante sobre decisiones concretas de modelado, sobre las propiedades verificadas, sobre las mutaciones del apartado 5 y sobre posibles extensiones. Además, a cada integrante se le pedirá una **modificación pequeña sobre vuestra propia especificación**. **La defensa no tiene nota propia**, pero es **condición necesaria para aprobar el proyecto**: quien no la supere no aprueba el proyecto, por mucha nota que tenga lo entregado, y pierde por tanto la parte de la asignatura que corresponde a este bloque.

Sobre el uso de asistentes de IA. El uso de IA está permitido, siempre y cuando sea instrumental. En caso de usarla, se recomienda el MCP para Maude.

Evaluación. La nota del proyecto se compone de:

Especificación en Maude (estructura, ecuaciones vs. reglas, comentarios, término de ejemplo)	25 %
Propiedades definidas (apartado 3)	10 %
Verificación: `search` y model checker LTL (apartado 4)	25 %
Mutaciones (apartado 5)	20 %
Optimización de reglas a ecuaciones	5 %
Memoria (coherencia, justificación de simplificaciones, profundidad del desarrollo)	15 %

Esta nota es de equipo, ponderada por el reparto de tareas declarado, que el profesor puede ajustar si la defensa no lo respalda. La defensa no aparece en la tabla porque no suma porcentaje: hay que superarla para aprobar el proyecto y se valora de forma individual, así que una defensa que muestre que un integrante no entiende lo que ha entregado puede dejarlo por debajo del resto de su equipo o directamente suspenso, con independencia del desglose anterior.